

Fundamentals of Data Science

Sanjit Prakash

250701684

CSE

Assignment

Reference Raw Table (Before Preprocessing Starts)

Student ID	Name	Age	Gender	Marks	Attendance	City
101	Ravi	20.0	M	85	90	Chennai
102	Priya	19.0	F	92	95	Chennai
103	Arjun	NaN	M	78	88	Madurai
104	Meena	20.0	F	450	91	Chennai
105	Ravi	20.0	M	85	90	Chennai
106	Kiran	21.0	Male	NaN	NaN	Caimbatore

Step 1: Data Profiling

1. Structure Discovery: Inspecting Schema & Data Types

Python Code:

```
df.info()
```

Function name: `Dataframe.info()`

Syntax Breakdown: `df.info()` analyzes memory usage, non-null counts, and data types.

What Data Profiling Reveals Here: Age, Marks, Attendance lack full non-null counts.

console Output:

2	Age	5	non-null	float 64 ← ... (1 missing)
4	Marks	5	non-null	float 64 ← ... (1 missing)
5	Attendance	5	non-null	float 64 ← ... (1 missing)

2. Content Discovery (Part A): Checking Missing Values (NULL)

Target Rows & Columns: Row 2 (Age), Row 5 (Marks, Attendance)

Python code:

```
print(df.isnull().sum())
```

Function Name: `DataFrame.isnull().sum()`

Syntax Breakdown: `.isnull().sum()` counts boolean True missing values per column.

What Data Profiling Reveals Here: 1 missing in Age, Marks, Attendance.

Why it is used: Pinpoints specific columns requiring imputation.

3. Content Discovery (Part B): Value Distribution & Outlier Detection

Target Rows & Columns: Row 5 (Gender), Row 3 (Marks).

Python code:

```
print(df['Gender'].value_counts())  
print(df['Marks'].describe())
```

Function Name: `Series.value_counts()` and `DataFrame.describe()`

Syntax Breakdown: Checks unique categorical frequencies and numerical summaries.

Console Output:

Male 1 <--- (Inconsistent label)

max 450.0 <--- (Domain Error : max > 100)

4. Relationship Discovery: Checking for Duplicate Profiles & Key Integrity

Target Rows & Columns: Checks all rows for redundant records.

Python code:

```
print(df.duplicated(subset=['Name', 'Age', 'Gender', 'City']).sum())
```

Function Name: DataFrame.duplicated()

What Data Profiling Reveals Here: 1 duplicate found (Row 4).

Step 2: Data Cleansing (Fixing Structural Errors and Formatting)

1. Standardizing Inconsistent Categorical Labels in Gender

Target Row & Column: Row 5 under Gender

Python code:

```
df['Gender'] = df['Gender'].replace({'Male': 'M'})
```

Function name: .replace()

Syntax Breakdown: Replaces incorrect string entries to uniform standard entries.

Why it is used: Steps categorical encoders from creating redundant dimensions.

Step 2: Removing the Duplicate Student Profile
Target Row & Column: Row 4 (Ravi).

Python code:

```
df = df.drop_duplicates(subset = ['Name', 'Age', 'Gender', 'City'], keep='first')
```

Function Name: `.drop_duplicates()`

Syntax Breakdown: Drops repeating matches based on specific profile columns.

Why it is used: Prevents double-counting a student's metrics.

Summary of changes in step 2:

1. Gender 'Male' standardized to 'M'.
2. Row 4 duplicate dropped.

Step 3: Constraint Validation (Domain Bounds)

Target Row & Column: Row 3 under Marks (Value: 450).

Python code:

```
df.loc[df['Marks'] > 100, 'Marks'] = np.nan
```

Function name: `df.loc[]`

Syntax Breakdown: Flags out-of-bounds metrics (Marks > 100) and resets them to Null.

Why it is used: Isolates the error without deleting the students' valid Name, Age etc.

4 : Missing Data Imputation (Nan Handling)

Target Rows & Columns : Age (Row 2), Marks (Row 3, 5), Attendance (Row 5).

Calculation : Median Age (20.0), Mean Marks (85.0), Mean Attendance (91.0)

Python Code :

```
df['Age'] = df['Age'].fillna(df['Age'].median())
```

```
df['Marks'] = df['Marks'].fillna(df['Marks'].mean())
```

```
df['Attendance'] = df['Attendance'].fillna(df['Attendance'].mean())
```

Function Names : .fillna(), .median(), .mean()

Syntax Breakdown : Calculates central tendency metrics and fills Null spots.

Why it is used : Preserves data less by logically estimating missing blanks.

Step 5: Data Transformation

Target Columns : Marks, Attendance, Age, City.

Python code :

```
df['Marks_per_Attendance'] = df['Marks'] / df['Attendance']
```

```
df['Age-group'] = df['Age'].cut(df['Age'], bins = [0, 20, 100], labels =  
    = ['Teen', 'Adult'])
```

```
region_map = { 'Chennai': 'North TN', 'Kumbhore': 'West TN', 'Medurai': 'South TN' }
```

```
df['Region'] = df['City'].map(region_map)
```

```
from sklearn.preprocessing import MinMaxScaler
```

```
df[['Norm-Marks']] = MinMaxScaler().fit_transform(df[['Marks']])
```

Function Names: `pd.cut()`, `map()`, `MinMaxScaler()`

System Breakdown & Why it is used: Derives ratios, groups continuous numbers into categories, reduces high-cardinality locations, and bounds numbers to $[0, 1]$ for ML stability.

Step 6: Data Reduction

Target Columns to drop: Student-ID, Name,

Python Code:

```
df_reduced = df.drop(columns=['Student-ID', 'Name'])
```

Function Name: `DataFrame.drop()`

System Breakdown and why it is used: Removes non-predictive identifiers to combat model overfitting.

Step 7: Data Enrichment

Target Rows & External Table: Join Col-Index onto City.

Python Code:

```
city_col_table = pd.DataFrame({'City': ['Chennai', 'Coimbatore', 'Madurai'], 'Col-Index':  
                                [1.2, 1.8, 0.9]})
```

```
df_enriched = pd.merge(df_reduced, city_col_table, on='City', how='left')
```